

BACKEND DEVELOPMENT PROGRESS

TECHNOLOGIES AND SERVICES UTILISED



- Spring Boot: A Java framework utilised for creating RESTful APIs and providing tools for implementing a user verification system.



- MySQL: Employed to implement a relational database for storing user login details.



- PlanetScale: Utilised for efficient MySQL database hosting to test various systems that are being implemented. This service is primarily being used for testing purposes and we can switch back to AWS to later on.

SETTING UP INITIAL BACKEND APPLICATION VIA SPRING BOOT

```
@Entity
public class User {

    @Id
    @GeneratedValue
    private long id;
    2 usages
    private String username;
    2 usages
    private String password;
```

Implementing Entity class "User"

@Entity indicates that the instances of the "User" class will be mapped into a table called "user".

@Id marks 'id' as the primary key.

String username & password will be persisted as columns of the corresponding table within the database.

```
@RestController
@CrossOrigin("http://localhost:3000")
public class UserController {

    @Autowired
    private UserRepository userRepository;

    // psyha7
    @PostMapping("/user")
    User newUser(@RequestBody User newUser){
        return userRepository.save(newUser);
    }

    //returns all the users
    // psyha7
    @GetMapping("/users")
    List<User> getAllUsers(){
        return userRepository.findAll();
    }
}
```

Implementing class "UserController"

@RestController indicates that this class is a 'RESTful' controller meaning it handles HTTP requests.

@CrossOrigin("http://localhost:3000") identifies requests from the frontend application's origin.

@PostMapping("/user") maps HTTP POST requests with the /user endpoint to the newUser method. It specifies that this method is responsible for creating a new user.

```

application.properties
1  spring.jpa.hibernate.ddl-auto=update
2  spring.datasource.url=jdbc:mysql://aws.connect.psdb.cloud:3306/evidenproject
3  spring.datasource.username=3ycq2eaz3co8e55801f9
4  spring.datasource.password=pscale_pw_oDcVJ5Z4QyDyEf8omenUTsCHapLsVi3iIYVQYrN0UuH
5  spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

```

Implementing Application Properties to connect to database.

'Spring.datasource.url' ensures Spring Boot knows where to connect to the database when the application starts. In this case, it is being connected to hostname provided by planetscale.

'Spring.datasource.username' & 'spring.datasource.password' are credentials used to access the database.

UPDATING LoginComponent WITHIN REACT (UI)

```

// Function to handle form submission
const handleSubmit = async (event) => {

  event.preventDefault();
  // Validation of username and password, setting login status in localStorage
  if ([username === 'admin' && password === 'admin']) {
    localStorage.setItem('isLoggedIn', 'true');
    // Redirect to dashboard upon successful login
    navigate('/dashboard');
  } else {
    // Show an alert for invalid credentials
    alert('Invalid username or password');
  }
  //Post data into database via AXIOS POST
  await axios.post("http://localhost:8080/user", { username, password })
};

```

Inside the handleSubmit function, added code to post data into database. In this case, when the username and password match 'admin', the data objects 'username' and 'password' will be sent to the end point 'http://localhost:8080/user', which in this case is where the spring boot application handles data.

RESULTS GATHERED:

Log-in Page

Username:

Password:



*Login with
username/password 'admin'*

Login

Sign Up

Forgot Password

```
1 SELECT * FROM user;
```

*Run an SQL query to retrieve data from
'user' Table within the database.*

	id	password	username
▶	102	admin	admin

*Password and username are shown with a
unique id (primary key)*

Testing and Documentation conducted by Humais.

NEXT STEPS:

Follow the plan created by Andreas.